浙江大学

Genai赋能工业编程

面向CODESYS的ST代码生成Agent

课程名称：GenAI赋能工业编程

学院专业：控制科学与工程学院

小组成员：待补充（五位成员姓名与学号）

指导教师：待补充

完成日期：2026年11月

成员分工

目录

1 大作业准备 **1**

1.1 项目思考与讨论 1

1.2 项目目标与主要工作 1

2 数据规范 **2**

2.1 数据集构成 2

2.2 公共题目转换 2

2.3 自建题流程 2

2.4 LTL Spec规约 3

2.5 共同的测试用例 4

3 自研Agent harness **5**

3.1 最开始的实现 5

3.2 问题改进 5

3.3 最终架构 6

3.4 生成、编译、修复与终止 8

3.5 Codex CLI 基线接入 9

3.6 自动化测试与批量运行 9

4 实验环境与公平实验设计 **10**

4.1 实验环境 10

4.2 模型、参数与四种设置 10

4.3 公平性控制 11

5 最终实验结果 **11**

5.1 总体结果 11

5.2 四种设置对比 12

5.3 模型框架结果分析 12

5.4 项目前后改进变化 14

6 结果分析 **15**

6.1 编译通过与功能分析 15

6.2 模型差异 15
6.3 框架差异 16
6.4 成功案例：public_20 16
6.5 语义失败案例：custom_06 16
6.6 组合差异案例：public_19 16
6.7 失败类型与局限 17

7 结论 **17**

A 电子附录与证据目录 **17**

B 报告数字追溯表 **17**

1 大作业准备

1.1 项目思考与讨论

根据学习，ST 是 IEC 61131-3 标准中的文本化 PLC 编程语言，适合表达状态保持、互锁、顺序控制、定时和数据处理等逻辑。大模型已经能够根据自然语言生成形式完整的程序。但课堂中也提到了PLC 任务与普通算法题存在明显差异：程序会被周期性扫描，输出常常依赖此前状态，停止、急停和故障还具有明确优先级。模型给出代码之后，仍需要真实工程环境确认接口、类型和调用链，再通过功能测试检查控制行为。

课程中关于软件测试的部分指出，如果测试只覆盖少数样例，即使全部通过，程序仍可能在未覆盖输入上出错。基于这个结论，我们适当地调整了本项目的评价方法。所以，我们将问题分成了三个层次，候选结构是否符合题目接口，代码能否通过 CODESYS 编译，运行行为能否通过需求对应的测试。这些也分别对应着后文的报告编译率、测试通过率和任务成功率等

1.2 项目目标与主要工作

课程作业要求是在全部公共题和不少于 10 道自建题上，对自研框架、现成 Agent 和两个基座模型进行四种设置比较。围绕这一目标，我们完成了项目的工作流程如下：

1. 将公共题和自建题统一为可执行的数据结构（比如json，初期我们也有采用md但是不是很方便读取），并写下自建题的来源、改编、审核和 PLC 合理性；
2. 完善数据部分之后，我们便逐题编写LTL Spec与测试用例，使自然语言需求可以追踪到具体性质和测试步骤；
3. 之后开始完善代码部分，打通模型、Python、CODESYS编译与Simulation自动测试，建立生成、检查、修复等的闭环，构建出一个agent harness；
4. 确保实验条件一致后完成 120 次正式运行，然后从模型、框架、难度和题库类型等角度分析结果。

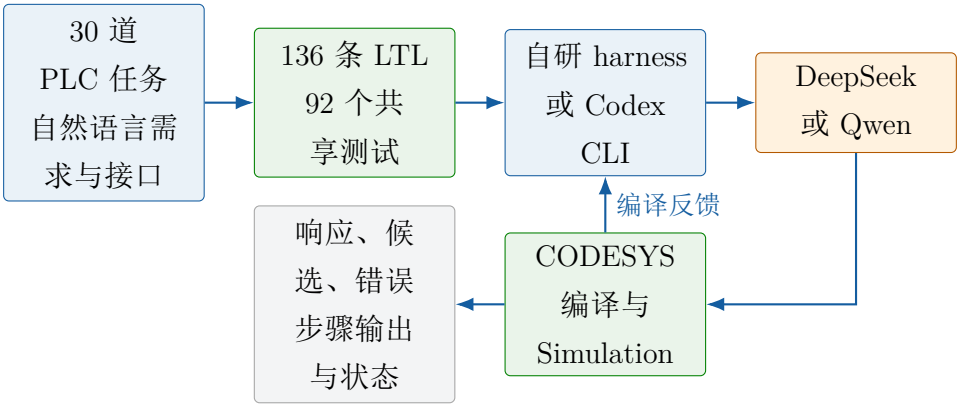


图 1　从需求到结果的关系

上面是我们的项目主线，首先题目与Spec定义输入和约束，然后Agent与模型产生候选，再之后CODESYS 提供真实编译和测试结果，为了对比公平性，我们的所有中间产物会进入一个原始证据目录。报告中的数字由这些记录重新汇总生成，可以更加方便对比和查看。

2 数据规范

2.1 数据集构成

项目数据集包含老师提供的 20 道公共题和 10 道自建题。公共题采用课程提供；自建题基于ai和我们的检索补充了安全锁存、报警复位、批次流程、定时监控和现场联锁等。最终规模见表 1。

表 1 数据集规模

类型	题目数	LTL性质	测试用例	测试步骤
公共题	20	92	62	263
自建题	10	44	30	89
合计	30	136	92	352

30道题按统一标准分为easy 9道、medium 12道和hard 9道。

2.2 公共题目转换

保持原始材料不变，但是实验使用转换后的JSON，这样可以每道题明确任务ID、需求、POU名称与类型、公开接口，以及难度和必要的编译上下文，转换之后可以解决原始题面与自动实验之间的接口衔接。

其中，使用DUT、数组和`VAR_IN_OUT`的题目需要在临时工程中创建固定类型并绑定实例；`public_18`的FIFO容量统一为10，元素类型统一为DINT，四组实验都使用同一个版本。

2.3 自建题流程

题目构建结合老师提供的公共题，遵循相同过程。先从现场控制场景提取一个边界清晰的功能任务，再确定公开输入输出和初始状态，补充急停、故障、复位、保持或最小时间等规则，最后由题面、Spec和测试交叉审核。

在题目生成后，我们进行微调，在`custom_01`中强调急停锁存与人工复位，`custom_06`包含填充、混合、排空的批次状态推进，`custom_09` 关注输送机在时间窗口内的堵料监

测。这些任务能够用 PLC 扫描周期和现场信号自然表达，也覆盖了从布尔控制到多阶段时序逻辑的变化。由于我们的自编题接口、定义可能更加规范，所以后面在实验的时候自编的数据会表现得更好。具体10道题的来源与逐题说明在后面补入附录。

表 2　选取部分自编题说明

题目	场景	考虑难点	人工审核修改点
custom_01	设备安全锁存	安全优先级	急停解除后不得自动恢复，必须人工复位
custom_06	配料批次流程	多阶段状态机	填充、混合、排空顺序与阶段退出条件完整
custom_09	输送机堵料监控	定时与复位	连续异常达到时间窗口才报警，并明确恢复条件

2.4　LTL Spec规约

自然语言题面便于阅读，但安全优先级、保持条件和扫描后的状态变化确实很容易产生不同理解。为此，我们每道题先定义命题，然后再按安全、响应、保持、优先级和不变量等类型写成LTL。

在本项目中，使用与任务直接相关的基本算子：G 表示性质始终成立，X 表示当前输入经过一次 PLC 扫描后的状态。

这里以高低限报警题 public_03 为例，题目要求高报警越过上限时置位，只有低于上限减滞回窗口后才清除；配置错误还要优先返回对应状态码。

表 3给出需求、性质与测试的对应关系。具体30道题的逐条LTL将在下一步整理到电子附录

表 3　public_03的需求、LTL和测试对应示例

需求	LTL性质	测试覆盖
上下限配置非法时优先报错	$G(invalid_levels \rightarrow X(error \wedge status_8001))$	配置错误优先级
越过高限后触发高报警	$G((valid \wedge above_hi) \rightarrow X(hi_alarm))$	高报警置位
进入清除区后清除报警	$G((valid \wedge below_hi_clear) \rightarrow X(\neg hi_alarm))$	滞回保持与清除
合法配置返回成功状态	$G(valid \rightarrow X(\neg error \wedge status_ok))$	正常配置状态码

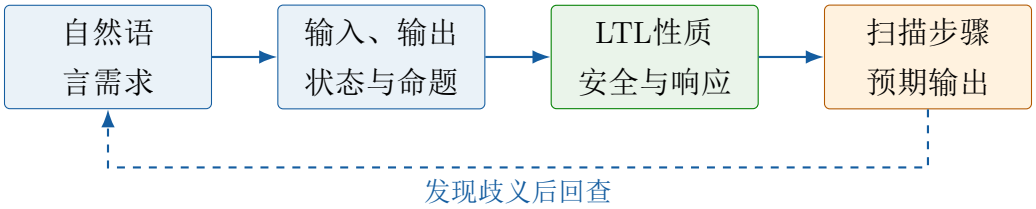


图 2　需求、LTL与测试之间的追踪关系

2.5　共同的测试用例

我们的每个测试用例参考word文件，包含初始状态、输入步骤、等待时间、预期输出和覆盖内容。执行器只通过公开输入建立状态，不直接改写候选内部变量；每一步写入输入后等待 PLC 扫描，再读取输出并比较。之后，正常流程、冲突输入、安全优先级、状态保持、复位和时间边界均在题目需要时纳入测试。

同一道题的四种设置使用同一份测试和判定标准。测试内容不提供给 Agent，特别是codex，这样能够避免候选针对答案进行拟合，codex作为一个比较成熟的agent框架，有必要进行约束。在编译失败时，题目计划测试全部计为未通过。LTL在本项目中用于整理需求并进入最终框架的任务上下文。项目暂时没有接入PLCVerif，只测试过一个自建的ST代码解释器，所以后面不报告形式化验证通过率。

3 自研Agent harness

3.1 最开始的实现

我们的项目先实现最小闭环，读取一道题，然后调用模型生成ST，写入临时工程，启动CODESYS编译；若编译失败，则把错误反馈给模型，之后最多修复两次。这一步确认了模型API、候选格式和CODESYS外部边界能够接通，但后面我们在使用的时候也发现三个问题。

第一，部分候选的POU名称、类型或公开接口明显错误，仍会消耗一次CODESYS启动。

第二，自然语言题面对保持、优先级和状态变化的表达不够稳定，编译通过后的功能失败较多。

第三，编译与测试分别启动CODESYS，而测试脚本本身还要重新编译工程，形成重复开销。

3.2 问题改进

针对上述问题,我们不断迭代,最终实现增加了三个直接改动。候选进入CODESYS前先检查POU名称、类型、声明头和公开接口；逐题LTL Spec与自然语言需求一起提供给两个框架；编译与共享测试合并为一次CODESYS执行。最后正确率有所提高，同时也能减少中间修改带来的运行时间增加。

表 4　问题和解决

开发中观察到的问题	修改	设计目的
明显接口错误仍启动外部工具	增加本地候选预检	提前结束确定无效的候选
编译通过后存在语义错误	将逐题LTL加入共享上下文	明确安全、保持和状态变化要求
编译和测试重复启动CODESYS	合并为一次执行	减少外部进程启动和重复工程加载
测试可能泄露预期行为	测试结果不进入修复上下文	保持评价独立与四组公平

3.3 最终架构

最终的harness保持少量模块和直接数据流。任务入口读取题目、LTL和共享测试；单题循环负责提示词、候选次数和终止条件；模型客户端只处理API请求；候选模块完成JSON解析和接口预检；CODESYS边界负责复制工程、编译、Simulation测试和结果读取；最后再由汇总脚本生成CSV、JSON、表格和图片。图 3先给出简要数据流，图 4是具体的函数功能相关的架构。

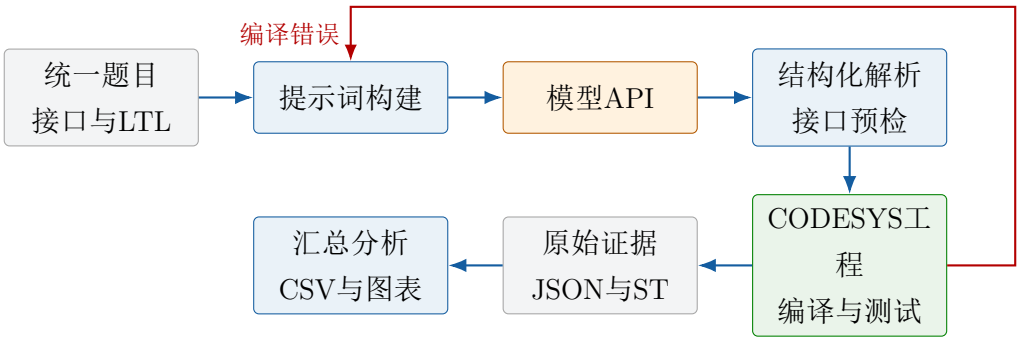


图 3　最终harness的简要数据流

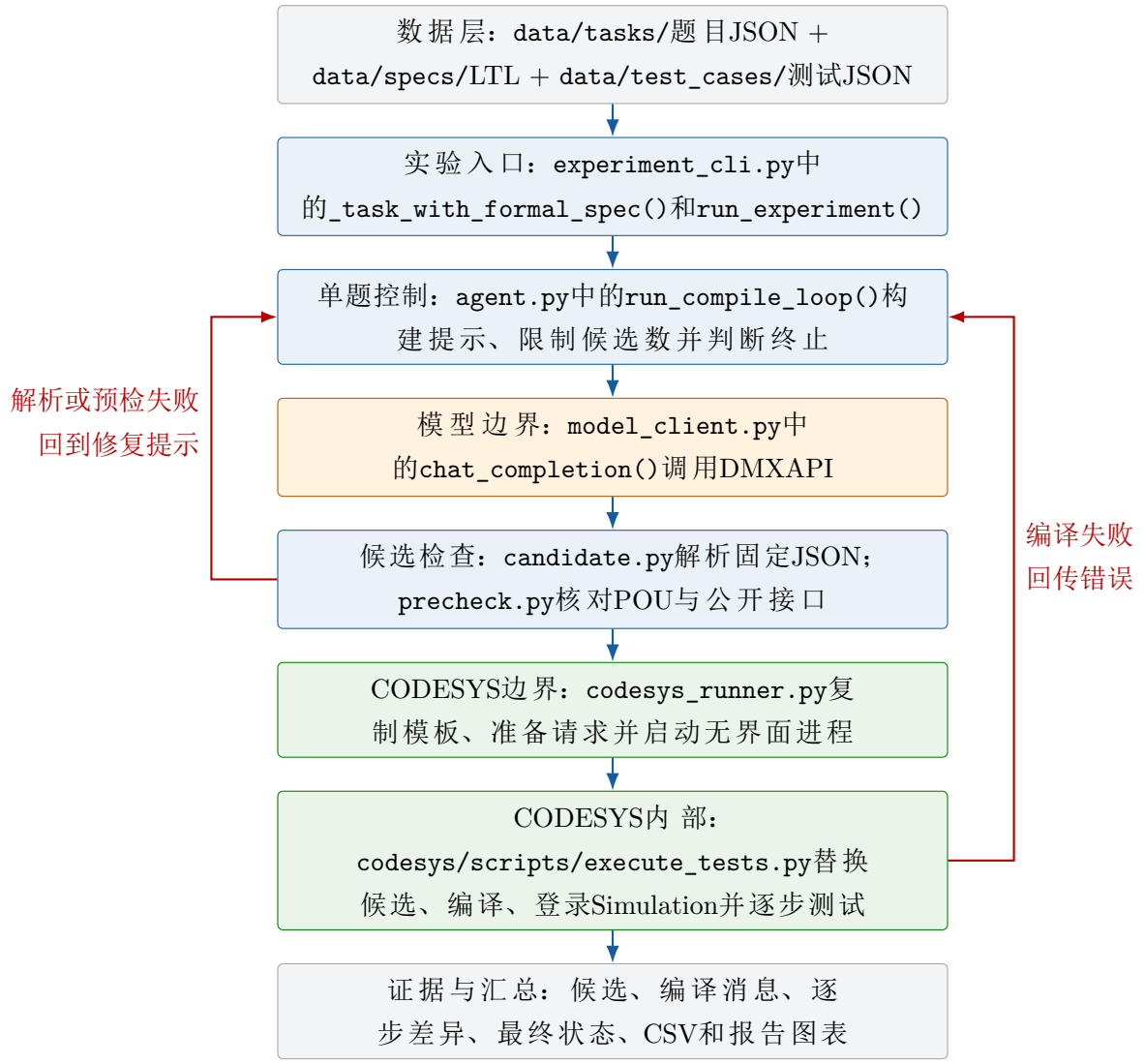


图 4　harness架构

架构图中的每一层都能在代码里找到直接入口。表 5列出主要文件和函数

表 5　主要功能、Python文件与函数的对应关系

功能	文件与函数	实际职责
四组实验入口	`experiment_cli.py: run_experiment()`	组合题目、框架和模型，持续写入运行清单
拼接LTL上下文	`experiment_cli.py: _task_with_formal_spec()`	将命题、假设和性质放入同一道题的模型上下文
单题闭环	`agent.py: run_compile_loop()`	控制生成、修复、编译、测试和最终状态
提示词渲染	`agent.py: _render_prompt()`	填入任务、上一候选和编译错误
模型调用	`model_client.py: chat_completion()`	调用OpenAI兼容接口并保存原始响应与Token
候选解析	`candidate.py: StCandidate.from_model_text()`	限定四个JSON字段、POU类型和声明内容
接口预检	`precheck.py: validate_candidate()`	核对POU名称、类型、声明头和公开变量
测试文档检查	`codesys_runner.py: validate_test_document()`	禁止测试读写非公开接口，检查步骤和等待时间
CODESYS启动	`codesys_runner.py: execute_tests()`	复制模板、写请求文件并启动CODESYS脚本
工程包装	`execute_tests.py: replace_candidate()`	创建功能块实例和`PLC_PRG`调用链
Simulation测试	`execute_tests.py: run_tests()`	冷复位、写输入、等待扫描、读输出并记录差异
输出比较	`execute_tests.py: values_equal()`	比较BOOL、数值、时间、十六进制、数组和结构体值

3.4　生成、编译、修复与终止

初次请求包含需求、公开接口、必要编译上下文和LTL Spec。模型按固定JSON格式返回`pou_name`、`pou_type`、`declaration`和`implementation`。

在解析成功且接口预检通过后，候选功能块会在临时`PLC_PRG`中实例化并调用，确保代码真正进入编译链。

如果不幸编译失败，系统会提取错误对象、位置和文本，下一轮增加上一候选与编译错误。每题最多产生三个候选；编译成功后直接进入共享测试，测试结束即终止。如果候选次数达到上限或模型调用出现不可恢复错误时也会终止，比如网络断联，这时候会保存原因。

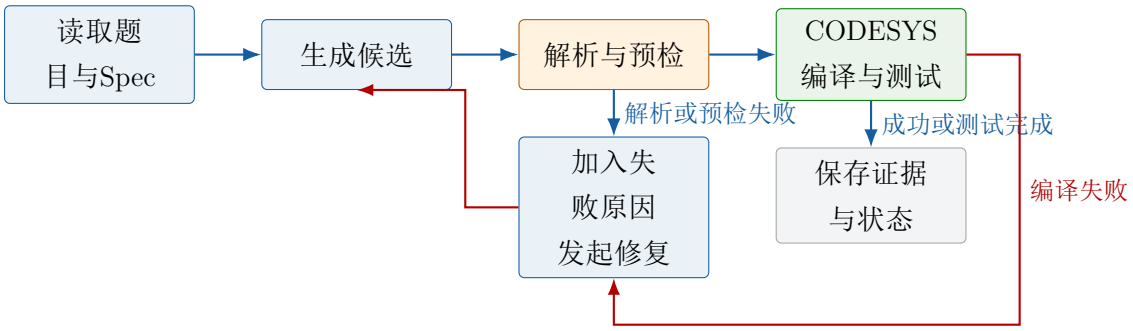


图 5　生成、检查、修复和终止流程

这一流程学习课程中的propose–check–repair思路，大致流程一致。首先模型提出候选，然后本地预检与CODESYS完成检查，编译错误驱动有限轮的修复。功能测试作为独立评价，在生成过程结束后给出最终判定。

3.5　Codex CLI 基线接入

现成 Agent 基线采用codex cli。开始时我们使用的是hugging face平台上面的smolagent，考虑到它比较简单，同时结构清晰也便于对比，但是使用之后发现和模型的协作能力实在太差，不能够和我们自己的harness进行完好的对比，遂放弃。之后采用推荐的codex cli。

Codex CLI与自研harness使用相同模型、题目、Spec、候选上限和CODESYS执行器。Codex CLI负责产生结构化候选，外部Python运行器把上一候选和编译错误交给下一轮。两个框架最终都输出同一数据结构，便于对比和分析。

3.6　自动化测试与批量运行

自动化测试保持为四个步骤。第一，Python读取逐题测试JSON，检查测试只能通过公开接口读写，并为当前候选复制独立的CODESYS模板工程。第二，CODESYS脚本写入候选；功能块任务会自动生成`PLC_PRG`调用链，然后重建整个工程。第三，编译通过后进入Simulation，每个用例先执行Cold Reset，再按步骤写入输入、等待扫描并读取公开输出。第四，执行器比较预期值和实际值，将编译消息、用例通过数和首个失败步骤写入`test_results.json`

正式实验从项目根目录启动。运行前在本地`.env`中配置模型接口，并确认CODESYS没有被其他进程占用，然后执行：

```
D:\python\python.exe -m src.plc_agent.experiment_cli --phase formal
```

程序读取`config/experiment.json`，依次组合30道题、两种框架和两个模型，共执行120个条件。终端逐题显示题号、框架、模型、最终状态、编译结果、测试通过数、耗时和证据目录。原始响应、候选ST、编译消息、测试结果、配置快照和运行清单会自动

保存到`experiments/raw/formal/`下新建的时间戳批次目录。如果运行中断，在同一命令后增加`--resume`并传入该批次目录继续，已完成的不会重复执行。

4 实验环境与公平实验设计

4.1 实验环境

正式实验在同一台计算机上顺序执行，环境见表 6。这里把硬件、软件、两个基座模型、两种Agent框架和调用限制放在一起，读者可以直接看到完整运行条件。模型通过DMXAPI接入，PLC代码在CODESYS Simulation中运行，Codex CLI版本号来自正式实验配置。

表 6　正式实验环境

项目	配置
操作系统	Microsoft Windows 11家庭中文版，10.0.26200
处理器与内存	AMD Ryzen 7 7840H，15.2 GB可用物理内存
Python	3.12.4
PLC开发环境	CODESYS V3.5 SP22 Patch 4，Simulation
基座模型A	`deepseek-v4.1-flash-1017`
基座模型B	`qwen3-coder-flash`
现成Agent基线	Codex CLI 0.167.0-alpha.3.2
自研Agent框架	Python harness、候选预检、编译修复和自动测试
模型接入方式	DMXAPI的OpenAI兼容HTTP接口
资源与时间限制	单机顺序运行；模型请求300 s，最多2次修复，测试进程300 s

4.2 模型、参数与四种设置

模型A为`deepseek-v4.1-flash-1017`，模型B为`qwen3-coder-flash`。单次请求超时为300秒，每题允许一次首次生成和最多两次修复。自动测试默认等待50 ms，以覆盖PLC扫描。

表 7　四种正式实验设置

Agent框架	DeepSeek	Qwen
Codex CLI	30道题	30道题
自研harness	30道题	30道题

4.3　公平性控制

输入阶段，同一道题使用相同需求、公开接口、编译上下文和LTL Spec；执行阶段，两个框架使用相同模型、候选次数、超时和CODESYS执行器；判定阶段，四组使用相同测试，测试内容不向Agent开放。模型质量失败、编译失败和功能测试失败按真实结果保留，在原始记录能证明SSL、连接重试或执行器故障时允许重跑。

每次运行都保存配置快照、提示词、模型原始响应、候选ST、编译消息、测试步骤输出和最终状态。便于我们回看分析数据

5　最终实验结果

5.1　总体结果

120次正式运行全部产生有效结果，90次首次候选编译通过，修复后共有96次通过编译；这96次中，53次通过本题全部测试，另外43次存在至少一个功能测试失败。总体测试通过数为205/368。

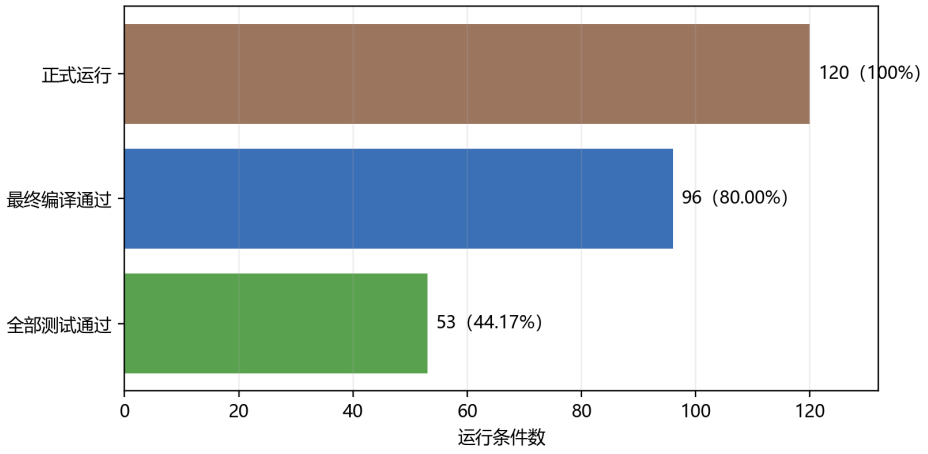


图 6　120次正式运行的结果

由图 6 可得首次编译率为75.00%，最终编译率为80.00%，测试通过率为55.71%，任务成功率为44.17%。最终编译率与任务成功率相差35.83个百分点，这个差距值得后面分析。

5.2 四种设置对比

表 8 四种设置的正式实验结果

框架	模型	首次编译	最终编译	测试通过	任务成功	平均修复	平均耗时/s
Codex CLI	DeepSeek	96.67%	100.00%	77.17%	63.33%	0.033	129.37
Codex CLI	Qwen	50.00%	56.67%	39.13%	30.00%	0.967	129.43
自研harness	DeepSeek	93.33%	100.00%	69.57%	53.33%	0.067	119.52
自研harness	Qwen	60.00%	63.33%	36.96%	30.00%	0.767	80.02

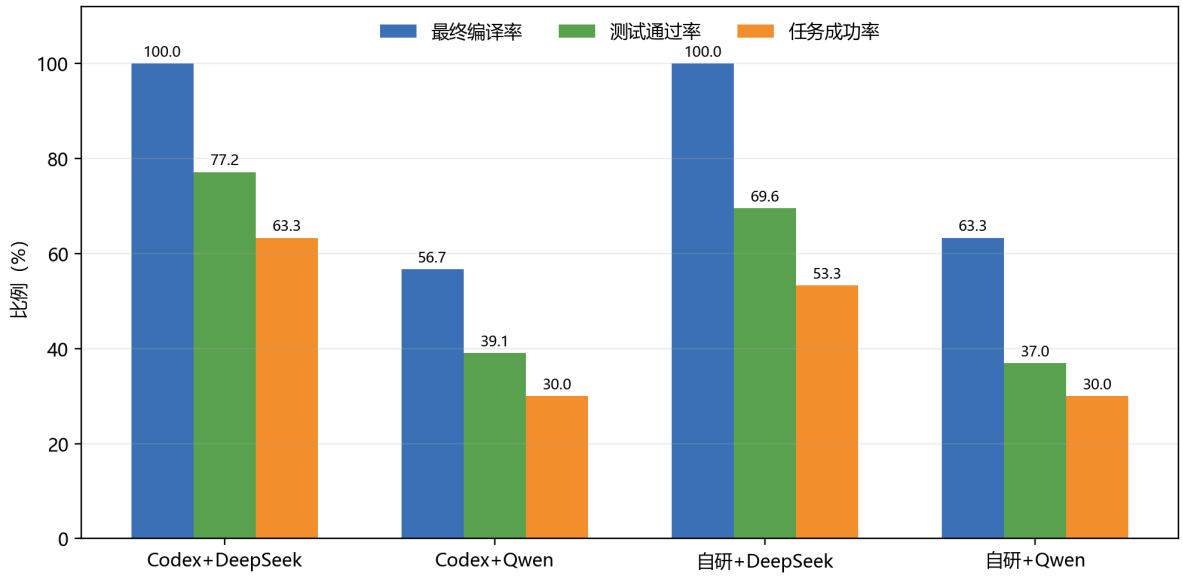


图 7 四种设置的最终编译率、测试通过率与任务成功率

从上面的结果可以看到，Codex CLI + DeepSeek 的三项核心指标最高，不愧是国产模型之光和现在最常用的agent架构呀!它们的任务成功率为 63.33%；自研 harness + DeepSeek 的任务成功率为 53.33%。两个 Qwen 设置的任务成功率均为 30.00%，但自研 harness + Qwen 的最终编译率略高，平均耗时较低。

出于api费用等考虑，我们没有继续搜索比deepseek效果高得多的模型，在最后比较合适的价位找到的qwen，它的效果会差一点，但是也可以作为一个对比，能够发现不是很优秀的模型在我们的项目架构改进的过程中，会发生什么变化。

5.3 模型框架结果分析

合并框架数据后，DeepSeek在60次运行中成功35次，任务成功率为58.33%，测试通过率为73.37%；Qwen成功18次，任务成功率为30.00%，测试通过率为38.04%。合并模型

后，Codex CLI成功28/60，自研harness成功25/60，对应46.67% 和41.67%。模型之间的差距明显大于框架之间的总体差距。

表 9　按难度和题库类型汇总的结果

分组	任务成功率	测试通过率
easy	61.11%	61.11%
medium	33.33%	51.39%
hard	41.67%	56.03%
公共题	36.25%	48.79%
自建题	60.00%	70.00%

hard在本次实验中高于medium，可能这与具体题目组成和需求表达有关。在前有论述，自建题的需求边界、接口和审核记录更统一，阐述更加规范，所以其总体结果高于公共题。



图 8　30道题在四种设置下的结果（绿：成功；橙：测试失败；红：编译失败）

热力图显示，`custom_01`、`custom_02`、`custom_05`、`public_01`、`public_03`和`public_20`在四组中均成功

5.4　项目前后改进变化

为了观察项目改进带来的整体变化，我们比较了两个覆盖相同120个条件的完整实验。该比较同时包含上下文、预检和执行方式等系统改进，用于展示迭代结果

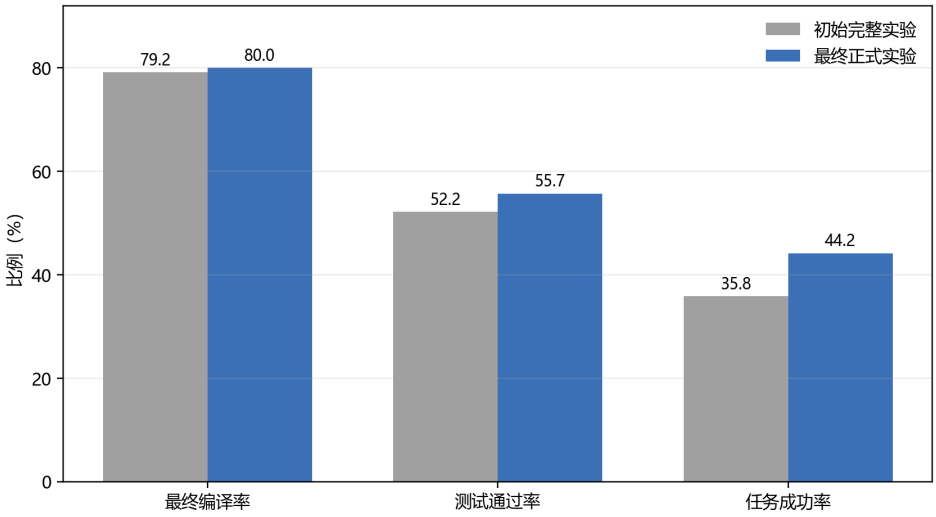


图 9　初始完整实验与最终正式实验的核心指标

任务成功数由43增加到53，测试通过数由192增加到205，最终编译数由95增加到96。对应地，任务成功率提高8.34个百分点，测试通过率提高3.54个百分点。配对比较中有19个条件的终态改善，92个不变，9个下降。

合并编译与测试后进行查看，原始目录中的CODESYS编译/测试进程记录由271次减少到170次，下降37.27%。端到端平均任务耗时由111.25 s增至114.59 s，增加约3.00%。外部进程启动减少，但模型响应波动、候选差异和更完整的上下文共同影响了最后执行的时间。因此本次优化主要改善执行结构，但是没有形成稳定的总耗时下降。

6　结果分析

6.1　编译通过与功能分析

96个条件通过最终编译，其中43个没有通过本题全部测试。这些候选已满足ST语法、类型、接口和工程调用链，却在部分需求上不一致。编译器解决的是程序能否进入PLC工程，功能测试回答程序在指定场景中做得是否正确，这也是课堂上说的语法和语义的区别。两类证据共同使用，才能判断任务是否完成。

测试通过率55.71% 高于任务成功率44.17%，也说明不少失败任务并非全部逻辑错误，而是在部分场景中出现偏差。逐步骤记录能够定位第一个不一致的扫描周期，为后续分析提供依据。

6.2　模型差异

DeepSeek在两个框架下均达到100% 最终编译率，Qwen合并后的最终编译率为60%。两者的测试通过率分别为73.37% 和38.04%。从逐题结果看，Qwen在复杂接口、多模式状态推进和较长上下文下更容易出现编译或功能问题。因此，在本实验的数据集和参数

范围内，基座模型能力是影响最终结果的主要因素之一。

害怕额度花完，我们也进行了能够查看范围内的token记录。Token记录也呈现出不同调用特征，Codex CLI + Qwen 的总 Token 为 3,248,894，明显高于其他三组；自研 harness 的两个模型合计 Token 较少。Codex的agent架构可能会引入更多的上下文和更多请求，增加token消耗。

6.3 框架差异

Codex CLI的总体任务成功率为46.67%，自研harness为41.67%，相差5个百分点。即使是换用很好的框架，也不能非常大幅提高成功率。DeepSeek下Codex CLI高10个百分点，Qwen下两者任务成功率相同；自研harness + Qwen的最终编译率高6.66个百分点，框架影响会随模型和题目变化。

自研harness的价值还体现在实验控制能力，它能够更加符合我们的需求。任务上下文、候选格式、修复上限、外部工具调用、测试隔离和证据目录都有明确边界，它没有依赖多Agent编排或复杂插件系统，主要模块可以直接追踪到课程要求中的生成、编译、修复、测试和公平比较。

下面对几个结果进行分析：

6.4 成功案例：public_20

public_20是五工位生产控制任务，难度为hard，包含多工位状态和协同控制。四种设置均通过编译并通过4/4测试。该结果说明，当题目的模式边界、接口和状态推进足够明确时，不同框架和模型都可能形成可执行的完整控制逻辑。

这个案例也解释了hard组结果没有低于medium的一部分原因，难度标签描述任务包含的结构，但题面表达质量和测试路径同样影响模型表现。public_20虽然状态较多，流程关系相对清晰。

6.5 语义失败案例：custom_06

custom_06描述填充、混合、排空三个阶段。四种设置全部通过编译，但没有一组通过全部测试；四组共通过6/12个测试。候选能够建立基本状态机，却会在阶段退出、计时完成或复位后的状态上产生偏差。

这个案例说明了编译通过不等于功能正确。若实验只记录编译结果，四组都会被判为成功；加入共享测试后，才能发现多阶段任务中的控制语义问题。

6.6 组合差异案例：public_19

public_19包含手动、单步、单周期和连续模式。四种设置都通过编译，只有Codex CLI + DeepSeek通过全部4个测试，其余三组存在功能失败。该题需要同时处理模式互

斥、状态推进和复位，模型与框架组合会影响上下文理解和最终控制结构。

6.7 失败类型与局限

这套实验仍有几项局限。首先，测试运行在CODESYS Simulation中，没有覆盖真实PLC的扫描周期波动、物理I/O延迟、通信故障和硬件资源限制。其次，92个测试用例和352个步骤覆盖了主要流程、边界和安全场景，通过测试只能说明候选满足当前测试集合。第三，LTL目前用于整理需求、检查测试覆盖并补充模型上下文，我们还没有接入PLCVerif等模型检查工具。最后，每个题目与设置只保留一次正式运行，但是模型输出具有随机性。

7 结论

本项目围绕PLC ST代码生成建立了完整、可执行的实验链路。数据部分包含30道题、136条LTL性质、92个测试用例和352个测试步骤；系统部分实现了自研harness、Codex CLI基线、两个模型接入、CODESYS编译和Simulation自动测试；实验部分完成了四种设置共120次正式运行，并保存从原始响应到逐步骤测试结果的证据。

项目改进围绕真实问题逐步推进。本地预检能够减少无效外部调用，LTL上下文补充语义约束，编译与测试合并减少重复CODESYS启动。

最终实验的首次编译率为75.00%，最终编译率为80.00%，测试通过率为55.71%，任务成功率为44.17%。DeepSeek的总体表现明显高于Qwen，两个框架的总体差距相对较小。96个可编译条件中有43个仍存在功能失败，说明PLC代码生成的主要难点已经从语法完整延伸到状态、优先级和时序语义。

A 电子附录与证据目录

完整题目位于`data/tasks/`，逐题LTL位于`data/specs/`，共享测试位于`data/test_cases/`。最终正式实验原始证据位于`experiments/raw/formal/20261111_102220_v4_formal/`；报告使用的逐任务汇总、四组汇总和初始—最终对比分别保存在`experiments/processed/`下。本文中的表格和图片由`experiments/analysis/`中的脚本从上述JSON生成。

B 报告数字追溯表

报告内容	处理后数据	原始来源
四种设置结果	`formal_summary.json/csv`	最终正式批次120个运行目录
逐题终态与耗时	`formal_runs.json/csv`	每个条件的`run_result.json`
初始与最终对比	`initial_final_comparison.json/csv`	两个完整正式批次
报告分析表	`formal_analysis.md`	汇总脚本自动生成
图 7、8、9	`generate_report_figures.py`	处理后JSON